

UNIVERSITÀ DEGLI STUDI DI CATANIA
Anno Accademico 2024 - 2025
Corso di Laurea in Informatica
Prova scritta di Programmazione 1 e laboratorio (A-E / O-Z)
19/02/25

COGNOME e NOME: (IN STAMPATELLO)	
N. MATRICOLA:	

**Non sono consentiti formulari, appunti, libri e calcolatori; non è consentito comunicare con i colleghi; ogni mezzo di comunicazione elettronico deve essere tenuto spento.
Il test va sempre consegnato prima dell'uscita dall'aula.**

Per ciascuna delle seguenti venti domande indicare l'unica risposta corretta.
La prova si considera superata con almeno undici risposte corrette.

1) Il seguente frame di codice

```
#define Y ... // Y e' un qualunque intero maggiore di 1
int x = Y;
while(--x>0)
    printf("%d", x);
```

- ☐ produrrà Y iterazioni;
- ☒ produrrà Y-1 iterazioni;
- ☐ produrrà Y+1 iterazioni;
- ☐ contiene un errore;

2) Completare il seguente frame di codice con l'unica istruzione corretta:

```
struct info{
    char *data[10];
}I1;
// ..
???
```

- ☐ printf("%s", I1.data);
- ☐ printf("%s", *I1.data[rand()%10]);
- ☐ printf("%s", I1.*data[rand()%10]);
- ☒ printf("%s", I1.data[rand()%10]);
- ☐ printf("%s", I1->data[rand()%10]);

3) La seguente istruzione:

```
printf("%zd", sizeof(double));
```

- ☒ il corrispondente output dipenderà dalla piattaforma;
- ☐ il corrispondente output sarà 4;
- ☐ il corrispondente output sarà 8;
- ☐ il corrispondente output sarà 16;
- ☐ nessuna delle risposte precedenti è corretta;

4) Il seguente frame di codice:

```
for(float x = 0.1; x!=1.0; x+=0.1)
    printf("\n%f", x);
```

- ☒ produrrà un numero di iterazioni indefinito;
 - ☐ produrrà 9 iterazioni;
 - ☐ produrrà 10 iterazioni;
 - ☐ produrrà 11 iterazioni;
 - ☐ nessuna delle risposte precedenti è corretta;
-

5) Con riferimento al seguente frame di codice:

```
unsigned ** foo(){
    unsigned *A = malloc(sizeof(unsigned*)*10);
    // ...
    return &A;
}
```

- ☐ essendo il puntatore *A* una variabile locale, l'array sarà deallocato automaticamente con l'uscita dalla funzione stessa;
 - ☐ il puntatore restituito dalla funzione potrà essere usato solo come puntatore void;
 - ☒ il codice della funzione contiene errori;
 - ☐ nessuna delle risposte precedenti è corretta;
-

6) Completare la seguente istruzione in linguaggio C per l'inizializzazione di un array di double con numeri pseudo-casuali

```
unsigned L = 10, i=0;
double A[L+1];
???
```

- ☐ while(i<=L) A[++i] = rand()*1.0/RAND_MAX
 - ☐ while(i<L+1) A[i++] = rand()/RAND_MAX*1.0
 - ☐ while(i++<L+1) A[i] = rand()*1.0/RAND_MAX
 - ☒ while(i<L+1) A[i++] = rand()*1.0/RAND_MAX
-

7) Con riferimento al seguente frame di codice C:

```
char *str = "abcdefg";
str[5] = '*';
```

- ☐ a seguito della sua esecuzione, il sesto carattere della stringa sarà uguale al carattere '*';
 - ☐ a seguito della sua esecuzione, il quinto carattere della stringa sarà uguale al carattere '*';
 - ☒ produrrà warning del compilatore;
 - ☐ produrrà errore a tempo di esecuzione;
 - ☐ nessuna delle precedenti risposte è corretta;
-

8) È possibile implementare una matrice di $n \times m$ elementi di tipo `double*`, in linguaggio C mediante i seguenti passi:

- i) allocazione di un array di n elementi di tipo `double ***`
- ii) allocazione di `__n__` vettori riga di lunghezza `__m__` i cui elementi siano di tipo `__double**`
-

9) con riferimento alla struttura dati CODA implementata come lista dinamica:

- ☐ è necessario mantenere un unico puntatore all'elemento in coda;
 - ☐ l'inserimento di un elemento richiede una visita completa della coda a partire dalla testa;
 - ☐ la cancellazione di un elemento richiede il puntatore alla coda;
 - ☐ l'inserimento di un nuovo elemento richiede il puntatore alla testa;
 - ☒ nessuna delle precedenti risposte è corretta;
-

10) Scegliere l'istruzione corretta per la definizione e contestuale inizializzazione di una stringa `s`:

- ☐ `char *s[] = "my_string";`
 - ☒ `char s[] = {'m', 'y', '_', 's', 't', 'r', 'i', 'n', 'g', 0};`
 - ☐ `char s*[] = "my_string";`
 - ☐ `char *s[] = {'m', 'y', '_', 's', 't', 'r', 'i', 'n', 'g', 0};`
-

11) Completare il seguente frame di codice mediante inserimento di una singola istruzione:

```
struct record{
    char *str;
    //...
};
void fill(struct record *ptr, const char *source){

    __ptr->str=malloc(sizeof(*source));__

    strcpy( ptr->str , source);
}
```

12) Completare il seguente frame di codice con una delle quattro proposte elencate in seguito

```
void bar(unsigned K, unsigned L, int ***V){
    for(unsigned i=0; i<K; i++)
        for(unsigned j=0; j<L; j++)
            ??? = rand()%10000;
}
```

- ☒ `** (V[i]+j)`
 - ☐ `**V[i+j]`
 - ☐ `**V[i]+j`
 - ☐ `V[i][j]`
-

13) Sia `source.c` un file sorgente C. Indicare il comando apposito che permetta di ottenere, su standard output, il risultato della fase di pre-processing (output del pre-processore):

- ☐ \$ gcc source.c -o source.S
 - ☒ \$ gcc -E source.c
 - ☐ \$ gcc -c source.c
 - ☐ \$ gcc -c source.c -E source.S
-

14) Completare opportunamente il seguente frame di codice selezionando una delle opzioni indicate nel seguito:

```
void foo(char **V, int ns){
    for(int i=0; i<ns; i++){
        char *ptr = V[i];
        ???
    }
}
```

- ☐ while(++ptr) {printf("%c", *ptr);}
 - ☐ while(ptr++) {printf("%c", *ptr);}
 - ☐ while(*ptr) {printf("%c", *(ptr+1));}
 - ☒ while(*ptr) {printf("%c", *ptr++);}
-

15) Per il seguente frame di codice:

```
void foo(){
    static double alpha = 0.5;
    alpha*=alpha;
    printf("foo(), alpha=%f", alpha);
}
```

- ☐ il qualificatore `static` permette di re-inizializzare `alpha` ad ogni invocazione della funzione `foo`;
 - ☐ dopo n invocazioni della funzione `foo()`, ultimo valore di `alpha` mostrato sullo standard output sarà uguale ad `alpha * n`
 - ☐ dopo n invocazioni della funzione `foo()`, ultimo valore di `alpha` mostrato sullo standard output sarà uguale ad `alpha * (n-1)`
 - ☒ se una variabile locale è qualificata `static`, questa non potrà essere modificata all'interno della funzione stessa, ma solo inizializzata;
 - ☐ nessuna delle risposte precedenti è corretta;
-

16) Sia `W` il nome di un array monodimensionale precedentemente allocato nel segmento `STACK` (allocazione automatica):

- ☐ `W` sarà deallocato automaticamente con la terminazione del programma;
 - ☐ `W` potrà essere deallocato invocando la funzione `free()`;
 - ☒ `W` sarà deallocato a seguito dell'uscita, del flusso di esecuzione, dalla funzione in cui `W` è stato dichiarato/allocato;
 - ☐ nessuna delle risposte precedenti è corretta.
-

17) Nella codifica di un numero in virgola mobile che si basi sullo standard IEEE 754 a 32 bit (singola precisione):

- ☐ non vi è alcun bit che rappresenta il segno, il quale è rappresentato implicitamente nel campo esponente;
- ☐ la cosiddetta precisione P dipende dall'ampiezza (numero di bit) del campo esponente;
- ☒ la precisione P è uguale a 6;
- ☐ l'intervallo rappresentabile è uguale a $[-2^{32} - 1, 2^{31} - 1]$;
- ☐ nessuna delle risposte precedenti è corretta;

18) In corrispondenza della seguente istruzione:

```
long y = INT_MAX * INT_MAX;
```

- ☐ si avrebbe errore in fase di esecuzione;
- ☐ si avrebbe errore in fase di compilazione
- ☐ si avrebbe warning in fase di compilazione;
- ☒ nessuna delle precedenti risposta è corretta;

19) Completare il seguente frame di codice affinché si possano stampare K numeri pseudo-casuali compresi tra 20.0 e 30.0. Usare opportunamente la funzione di libreria `rand()` per l'espressione (2), ed una espressione booleana che faccia uso di i e K per l'espressione (1).

```
// sia K una costante positiva
unsigned i=0;
while( (1)  ____i++<k  )
    printf("\n %f", (2)  _____(double rand()/RAND_MAX)*0.1_____ );
```

20) Completare il seguente frame di codice per il calcolo del montante M di un capitale C dopo X anni ad un tasso T

```
#define C 1000.0
#define T 0.1
#define X 5
```

```
double M = C;
int i = 0;
```

```
while( ??? )
    M+=M*T;
```

- ☐ i++<=X
- ☐ ++i<=X
- ☐ i++<X
- ☒ ++i<X